Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3596519

PDF Download
3596519.pdf
12 March 2026
Total Citations: 8
Total Downloads: 810

Published: 14 July 2023
Online AM: 17 May 2023
Accepted: 05 April 2023
Revised: 28 March 2023
Received: 18 May 2022

Citation in BibTeX format

SURVEY

A Systematic Study on Reproducibility of Reinforcement Learning in Recommendation Systems

EMANUELE CAVENAGHI, Free University of Bozen-Bolzano, Bolzano, BZ, Italy

GABRIELE SOTTOCORNOLA, Free University of Bozen-Bolzano, Bolzano, BZ, Italy

FABIO STELLA, University of Milano-Bicocca, Milan, MI, Italy

MARKUS ZANKER, Free University of Bozen-Bolzano, Bolzano, BZ, Italy

Open Access Support provided by:

Free University of Bozen-Bolzano

University of Milano-Bicocca

A Systematic Study on Reproducibility of Reinforcement Learning in Recommendation Systems

EMANUELE CAVENAGHI and GABRIELE SOTTOCORNOLA, Free University of Bozen-Bolzano, Italy
FABIO STELLA, University of Milano-Bicocca, Italy
MARKUS ZANKER, Free University of Bozen-Bolzano, Italy and University of Klagenfurt, Austria

Reproducibility is a main principle in science and fundamental to ensure scientific progress. However, many recent works point out that there are widespread deficiencies for this aspect in the AI field, making the reproducibility of results impractical or even impossible. We therefore studied the state of reproducibility support on the topic of Reinforcement Learning & Recommender Systems to analyse the situation in this context. We collected a total of 60 papers and analysed them by defining a set of variables to inspect the most important aspects that enable reproducibility, such as dataset, pre-processing code, hardware specifications, software dependencies, algorithm implementation, algorithm hyperparameters, and experiment code. Furthermore, we used the ACM Badges definitions assigning them to the selected papers. We discovered that, like in many other AI domains, the Reinforcement Learning & Recommender Systems field is grappling with a reproducibility crisis, as none of the selected papers were reproducible when strictly applying the ACM Badges definitions according to our analysis.

CCS Concepts: • **General and reference** → **Evaluation**; **Validation**; • **Computing methodologies** → **Reinforcement learning**; • **Information systems** → **Recommender systems;**

Additional Key Words and Phrases: Reproducibility, ACM badges

ACM Reference format:
Emanuele Cavenaghi, Gabriele Sottocornola, Fabio Stella, and Markus Zanker. 2023. A Systematic Study on Reproducibility of Reinforcement Learning in Recommendation Systems. *ACM Trans. Recomm. Syst.* 1, 3, Article 11 (July 2023), 23 pages.
https://doi.org/10.1145/3596519

1 INTRODUCTION

Recommender Systems (**RSs**) are algorithms developed for helping users to find items of interest. The massive volume of information available on the web leads to the problem of information overload and, thus, increases the need for delivering effective and timely recommendations. The

This work was supported by the Open Access Publishing Fund of the Free University of Bozen-Bolzano.
Authors' addresses: E. Cavenaghi and G. Sottocornola, Free University of Bozen-Bolzano, Piazza Università, 1, Bolzano, Italy, 39100; emails: {ecavenaghi, gsottocornola}@unibz.it; F. Stella, University of Milano-Bicocca, Piazza dell'Ateneo Nuovo, 1, Milano, Italy, 20126; email: fabio.stella@unimib.it; M. Zanker, Free University of Bozen-Bolzano, Piazza Università, 1, Bolzano, Italy, 39100 and University of Klagenfurt, Universitätsstraße 65-67, Klagenfurt, Austria, 9020; email: mzanker@unibz.it.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
© 2023 Copyright held by the owner/author(s).
2770-6699/2023/07-ART11 $15.00
https://doi.org/10.1145/3596519

main idea behind these methods is to predict users' interests based on their feedback on past interactions with items in order to recommend new unseen items matching their preferences. As reported in [58] and [39], traditional techniques implementing RSs involve content-based filtering [52], collaborative-filtering [57], knowledge-based [7], and hybrid methods [6]. Nonetheless, as stated in [66] and [38], these methods suffer from many problems such as cold start, scalability, low-quality recommendations, and high computational demand. Other methodologies, like the one presented in [30], exploit **Deep Learning** (**DL**) [25], a framework based on artificial neural networks with several stacked layers. DL gained popularity in RSs, due to his ability to learn complex relationships hidden in the historical data and thus provide more effective recommendations. One example of a successful application to RSs is provided by the GRU4REC method presented in [32]. However, DL methods are usually non-interpretable and require a lot of data to work well, thus, their superiority over traditional techniques in RSs is still under debate [21, 56].

However, an RS is not asked to solve a prediction problem, but a sequential decision-making problem since we have to actively recommend items to users. This allows the recommendation problem to be modelled as a **Multi-Armed Bandit problem** (**MAB**) [59] or ***Markov Decision Process*** (**MDP**) [5], and thus to be solved with **Reinforcement Learning** (**RL**) [64]. MAB is a powerful framework that allows agents to solve sequential decision-making problems under uncertainty [60]. In the standard version, an algorithm has K possible actions (or arms) to choose from and T rounds (or timesteps). In each round, the algorithm chooses an arm, performs the associated action, and receives a reward for its choice. The reward is drawn independently from some fixed but unknown probability distribution, which depends only on the chosen arm. Thus, after each round, the algorithm observes the reward for the chosen arm, but not for the other arms. An extension of the MAB framework is the MDP framework, since, in MDPs, past choices influence the current state and thus the actions we can take and the rewards we can obtain. Formally, MDPs are defined as a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is the transition probability $P(s_{t+1} = s' | s_t = s, a_t = a)$, i.e., the probability of the state s' at time $t + 1$ given that the agent performs action a in state s at time t, $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function defined as $R(s_t = s, a_t = a) = \mathbb{E}[r_t | s_t = s, a_t = a]$ while γ is a discount factor. The RS problem can be mapped to MDPs in several ways. For instance, by associating states with the users' representation, actions to the recommendation of each item, and rewards to the user's feedback that we are interested in. For a complete introduction to RL in RSs refer to [2]. The goal of an RS algorithm is to maximize user satisfaction, which can be translated into maximizing the reward signal, or multiple reward signals like in [61].

Given the ability of RL algorithms to handle sequential decision problems, learn by a trial-and-error process and focus on delayed rewards, there are several RL approaches used in RSs. Specifically, the last aspect is particularly relevant in RSs, where a myopic approach, which disregards long-term effects, can lead to sub-optimal results [29]. Some approaches use basic RL algorithms such as Q-Learning [77, 91], Monte Carlo Tree Search [90], Tree-structured Policy Gradient [9], and *REINFORCE* with baseline [74], while others use DL with RL like Q-Learning with a **Generative Adversarial Network** (**GAN**) [14] or actor-critic methods [77, 78].

In recent years, several papers, published in Science [35], AAAI [27], and RecSys [21], have investigated the aspect of replication and reproduction of recent AI and Machine Learning research. [27], for instance, quantifies the state of reproducibility of empirical AI research by analysing a total of 400 research papers from the conference series IJCAI and AAAI. The work of [21] focuses on the DL application on RSs. One main reason behind the observed replicability and reproducibility problem is the lack of open-source artifacts, i.e., the authors rarely share the code used to perform experiments. Although the number of published artifacts has increased in recent years [27], it still remains rare to have access to them. Furthermore, even when the

source code is published, it is sometimes incomplete or does not work properly. In order to increase the replicability and reproducibility of computer science research, ACM introduced a taxonomy of badges [1] to assess different aspects of reproducibility. Specifically, ACM introduced three separate badges related to artifact review: (i) the *Artifacts Evaluated* badge is given if the associated artifacts have successfully completed an independent audit, (ii) the *Artifacts Available* badge is applied to papers in which associated artifacts have been made permanently available for retrieval, and (iii) the *Results Validated* badge is given to papers in which the main results of the paper have been successfully obtained by a person or team other than the author.

The goal of this article is to assess the reproducibility of recent literature on the application of RL in RSs. According to the ACM definition of reproducibility [1], we have to rely exclusively on the artifacts shared by the authors, they comprise: software systems, scripts used to run experiments, input datasets, raw data collected in the experiment, or scripts used to analyse results. To the best of our knowledge, the artifacts provided were trustable and consistent with what was described in the original papers, and it is out of the scope of this work to fix and/or to complete the original artifacts. We share Heil et al.'s [31] line of argument that *"..source code contains implementation details that a future author is unlikely to replicate exactly from methods descriptions.."* since *"..small deviations can lead to different behavior between the original work and the reproduced one."* Specifically, we want to address three main research questions:

— To what extent is recent research in RL and RSs field reproducible?
— Following the idea introduced in [27], to what extent are the papers' shared artifacts relevant for the paper?
— Which ACM badges could be assigned to the collected papers?

This article is organized as follows. In Section 2, we describe how the papers have been collected and the methodology followed to reproduce their results. Furthermore, we define several variables to investigate the most important aspects related to the reproducibility of a work. Section 3 gives a detailed description of our analysis for those works making publicly available some artifact. Section 4, summarizes the current status of reproducibility by using the previously defined variables and the ACM badges. Finally, we give some suggestions/recommendations for improving reproducibility.

2 RESEARCH METHOD

2.1 Collecting Papers

The first phase of our research concerned the collection of the literature about RL and RSs. To make sure that our work is not affected by selection bias regarding the choice of the papers, we included in our analysis all the papers cited by three recent surveys on RL in RSs, namely [2], [15], and [84]. Our main goal is to collect a (possibly) unbiased sample of papers in the RL&RSs field, thus we relied on more than one survey to (possibly) overcame the biases that might arise from a single work. For the sake of completeness, at the time of analysis, [2] and [15] have not yet been published and therefore non-peer-reviewed. Please note that in the meantime, these surveys have been published as [3] and [16], respectively. However, in order to be consistent with our methodology, we did not update the sample of analysed papers. In addition, the changed references are not large enough that they could change the conclusions of our analysis. Specifically, [2] performed an exhaustive search through important databases (including ACM digital library, IEEE Xplore, SpringerLink, ScienceDirect) and Google Scholar using the keywords: recommender systems/engine/platform, recommendation, collaborative filtering, reinforcement learning and deep reinforcement learning. Then, the authors also screened popular conferences in the field including RecSys, KDD, WWW, SIGIR, and WSDM. In our work, we considered all the papers

referenced in the three surveys, regardless of the venue (journal, conference, or workshop), that addressed the RS problem by proposing a new RL algorithm and that were published between 2015 and 2021. However, we chose to exclude from our analysis the papers that addressed the advertisement placement problem, since these have too many differences in constraints and objectives with respect to the standard recommendation scenario. At this stage, we collected a total of 60 papers.

In the second phase, we tried to collect all the artifacts necessary to reproduce the experiments proposed by the authors in the published works. With the term "artifact", we consider the dataset(s) used in the experiments, the source code used for the preprocessing of the dataset(s), the algorithm implementation, and the evaluation phase. We contacted each author of each selected paper by e-mail, asking them to provide all the artifacts built to perform the experiments. To make sure the authors had enough time to respond, we waited 30 days before proceeding to the next step.

In the third phase, we scanned all the papers and the papers' web page on the conference/journal website searching for a link to open-access artifacts (usually these are provided as GitHub[1] repositories). However, we marked as *without artifacts* two candidates: [76] because the shared implementation only implements a simplified version of the algorithm and [67] because the shared repository[2] did not report the implementation of the algorithm presented in the paper, but proposed a benchmark to evaluate RL algorithms applied to RSs. A summary of our search is reported in Table 1. At this stage, combining our searches and the authors' replies, we collected the artifacts for 13 papers out of the initial 60.

In Table 2, we report the 13 papers that proposed a new RL algorithm to solve the RS problem and that shared some artifacts related to the paper. In the following of the article, we show the results of our experiments where we tried to reproduce the experiments proposed in these papers using the shared artifacts.

2.2 Reproducibility Methodology

Firstly, a formal definition of the term *reproducibility* needs to be specified. According to *ACM Artifact Review and Badging—Version 1.1*, with the term reproducibility, we mean "obtaining consistent results using the same input data, computational steps, methods, and code, and conditions of analysis (i.e., using the same artifacts)". It should be noted that ACM, with version 1.1, swapped the definition of *replicability* and *reproducibility* with respect to the old version 1.0.

In [21], the authors reproduced the works by applying the same evaluation procedure used in the original papers, with the addition of new baselines. We followed the same strategy, using the artifacts provided by the authors to reproduce the same experiments presented in the original papers. However, we reproduced only the algorithms proposed, because we are interested in testing reproducibility, and we do not want to compare algorithms with new baselines. Furthermore, in our experiments, we used the hyperparameters reported in the original papers (if these are present), otherwise, we used the default values set in the artifacts (i.e., the algorithm implementation). We emphasize that providing the hyperparameters used in the experiments is essential to allow reproducibility. For this reason, running a hyperparameter optimization procedure is outside the scope of our work.

Following the idea introduced in [27], we define a set of variables that are good indicators of reproducibility. In our work, for each selected paper, we looked for these indicators (i.e., variables) in the article and in the artifacts provided.

[1]https://github.com/.
[2]https://github.com/fuxiAIlab/RL4RS.

Table 1. The Table Shows, for Each Selected Paper, the year and Venue of Publications

Year	Venue	With open access artifacts	Without Artifacts
2021	AAAI		[67, 76]
	WSDM		[11]
	Others		[17]
2020	SIGIR	[77, 89]	[33, 41, 69, 83]
	WSDM	[91]	[44]
	WWW	[50, 71]	
	Others		[13, 42, 45, 47, 72, 73, 75, 86]
2019	AAAI	[9, 81]	
	Arxiv		[36]
	KDD	[24]	
	SIGIR	[74]	
	WSDM	[10]	
	Others	[4, 14, 90]	[8, 22, 28, 40, 55, 79, 80, 82]
2018	Arxiv		[18, 46]
	KDD		[12, 68, 87]
	RecSys		[85]
	SIGIR		[62]
	WWW		[20, 88]
	Others		[37, 48, 51, 70]
2017	Others		[26, 34]
2016	Arxiv		[19, 49]
2015	Arxiv		[63]
	Others		[43, 65]
Total		13/60 (21.7%)	47/60 (78.3%)

We report in the third column the papers for which we could collect the artifacts and in the fourth column the papers for which artifacts were not open-source.

— **Dataset:** Is the dataset shared?
— **Pre-processing Code:** Is the preprocessing code for all datasets open-source?
— **Hardware Specifications:** Is the hardware setup used for the experiments shared?
— **Software Dependencies:** Is the software setup (i.e., dependencies) used for the experiments shared?
— **Algorithm Implementation:** Is the algorithm code open-source?
— **Algorithm Hyperparameters:** Are the hyperparameters used for the experiments reported?
— **Experiment Code:** Is the code to reproduce the proposed experiments open-source?

With respect to [27], we are trying to test the first degree of reproducibility, *"R1: Experiment Reproducible"*, that is defined as "the execution of the same implementation of an AI method executed on the same data". However, as mentioned earlier, we will use the term *reproducibility* according to the ACM definition.

3 REPRODUCIBILITY ANALYSIS

In this section, we report the results of our reproducibility experiments. We ran our experiments on a server provided by the *Free University of Bozen-Bolzano* with *Ubuntu 20.04* as operating system.

Table 2. The Table Reports the Papers Selected for Our Experiments about Reproducibility

Algorithm	Paper
IRecGAN [4]	Model-Based Reinforcement Learning with Adversarial Training for Online Recommendation
TPGR [9]	Large-Scale Interactive Recommendation with Tree-Structured Policy Gradient
Top-k [10]	Top-K Off-Policy Correction for a REINFORCE Recommender System
GAN-LSTM [14]	Generative Adversarial User Model for Reinforcement Learning Based Recommendation System
Exact-k [24]	Exact-K Recommendation via Maximal Clique Optimization
2-Stage [50]	Off-policy Learning in Two-stage Recommender Systems
KGPolicy [71]	Reinforced Negative Sampling over Knowledge Graph for Recommendation
PGPR [74]	Reinforcement Knowledge Graph Reasoning for Explainable Recommendation
RL4REC [77]	Self-Supervised Reinforcement Learning for Recommender Systems
HRL [81]	Hierarchical Reinforcement Learning for Course Recommendation in MOOCs
KGQR [89]	Interactive Recommender System via Knowledge Graph-enhanced Reinforcement Learning
FMTCS [90]	Reinforcement Learning to Diversify Top-N Recommendation
Dyna-q [91]	Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation

The hardware configuration is composed of: 192 GB of RAM, 2 GPU Volta V100 - 32 GB RAM and 2 CPU Xeon 4208. The cluster nodes are managed by *slurm*.[3] To manage the different package requirements of each paper, we used Anaconda[4] environments.

In Section 3.1, we list the papers for which we were unable to execute the experiments with the provided artifacts. Instead, for each paper with working artifacts, we propose a detailed analysis and a table with the number of exercisable and reproducible results. Exercisable means that we could execute the experiment to obtain a result, while reproducible means that we obtain a result that is inside the interval, e.g., standard error, reported in the original paper. For the papers that do not report such an interval, we consider a result to be reproducible if the difference w.r.t. the original result is less than the 5%, i.e., $\frac{|new-original|}{original} \cdot 100 \leq 5$. In our GitHub repository (https://doi.org/10.5281/zenodo.7876001), we report additional details for each paper, such as (i) the Anaconda environment, (ii) the procedure that we used to execute the experiments, and (iii) the results that we obtained for each exercisable paper.

Regarding the algorithm *KGPolicy*,[5] proposed at *WWW '20* in the paper *"Reinforced Negative Sampling over Knowledge Graph for Recommendation"* [71], we could not run the algorithm on the machine described above. However, we were able to run the algorithm on a different machine, but due to the high computational demand, we could not reproduce the proposed experiment. We want to underline that the errors obtained on our server were due to our problems, probably due to missing privileges to install all the necessary software.

[3]https://slurm.schedmd.com/overview.html.
[4]https://www.anaconda.com/.
[5]https://github.com/xiangwang1223/kgpolicy.

3.1 Non-Exercisable Papers

In this section, we report the list of the non-exercisable papers, i.e., the papers for which we could not execute the experiments of the original paper. For each paper, we report here a brief description of the cause that lead to the error(s) and a file named "error.txt" in our GitHub repository that reports the specific problems.

- *FMCTS*[6] - *"Reinforcement Learning to Diversify Top-N Recommendation"* [90]. We were unable to run the code because we got an error in the algorithm implementation related to an index out-of-bound exception. We tried to contact the authors, by e-mail and by opening an issue on the GitHub repository of the FMCTS project, to solve the issue, but we did not receive any reply.
- *TPGR*[7] - *"Large-scale Interactive Recommendation with Tree-structured Policy Gradient"* [9]. We could not run the experiments presented in the paper since the pre-processing code was not provided. We tried to contact the authors via e-mail, but we did not receive any response. However, we verified that the algorithm implementation worked on the demo data included in the repository.
- *HRL*[8] - *"Hierarchical Reinforcement Learning for Course Recommendation in MOOCs"* [81]. We were unable to reproduce the proposed experiment because we got an error in the data reading process. We asked the authors, but they did not provide any working solution.
- *Dyna-Q*[9] - *"Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation"* [91]. We were not able to run the algorithm due to a missing file in the artifacts. We asked the authors for clarification, but they did not reply.
- *KGQR*[10] - *"Interactive Recommender System via Knowledge Graph-enhanced Reinforcement Learning"* [89]. We were unable to reproduce the experiments proposed in the paper because we got an error in the algorithm execution related to the TensorFlow package. However, we were able to run the algorithm on a demo dataset.
- *Top-k*[11] - *"Top-K Off-Policy Correction for a REINFORCE Recommender System"* [10]. In this case, we could not try to reproduce the experiments of the paper because the experiments were performed online.

3.2 Off-policy Learning in Two-stage Recommender Systems

In *"Off-policy Learning in Two-stage Recommender Systems"* [50], published at *WWW '20*, the algorithm *Two-Stage IPS* is described. The authors propose to model the recommendation problem as an MDP and to solve it with a two-stage off-policy gradient method that explicitly takes the ranker model into consideration while training the candidate generation model. The authors claim that such a two-stage RL approach has the advantage of being efficient in selecting a relevant set of items for the user, while removing the bias inherited from the logging policy. The evaluation of the system is based on a semi-synthetic online simulation derived from two large publicly available multi-labelled datasets.

The datasets used in the experiments in the original paper are the MovieLens-1M and the Wiki10-31k. Both are publicly available, and we were able to download them from the links provided in the paper. However, in the shared artifacts, only the preprocessing of the MovieLens dataset is

[6] https://github.com/zoulixin93/FMCTS.
[7] https://github.com/chenhaokun/TPGR.
[8] https://github.com/jerryhao66/HRL.
[9] https://github.com/zoulixin93/pseudo_dyna_q.
[10] https://www.dropbox.com/sh/knebi6y3lcvwv9x/AADsTZnF2dpm7xoA7n4PnN0Oa?dl=0.
[11] https://github.com/awarebayes/RecNN.

Table 3. Number of Exercisable and Reproducible Experiments for Paper *"Off-policy Learning in Two-stage Recommender Systems"* [50]

	Exercisable	Reproducible
Proposed Algorithm(s)	85.7% (12/14)	21.4% (3/14)
Baselines	85.7% (24/28)	7.1% (2/28)
Total	85.7% (36/42)	11.9% (5/42)

provided, so we were unable to include the Wiki10-31k dataset in our analysis. Following our request via e-mail, the authors did not reply with the missing preprocessing code. The hardware and software configurations are not reported in the paper, but, in the GitHub repository,[12] the authors shared a configuration file (.yml file) to build the Anaconda environment with the packages necessary to execute the code. The values of the following hyperparameters are reported in the paper: learning rate, sample size, c_1, c_2, and α. However, we were able to set only the learning rate and α. We asked the authors how to set all the hyperparameters, but, in the reply, they stated that the default values were acceptable.

In Table 3, we report the number of experiments exercisable and reproducible for the experiments reported in the paper. We could execute all the experiments on the MovieLens dataset, for both the baselines and the proposed algorithm, respectively, 24 out of 24 and 12 out of 12 results. However, we could not execute the 4 and 2 experiments, respectively, for the baseline and the proposed algorithm, on the Wiki10-31k dataset therefore we obtain the 85.7% (36/42) of executable experiments. Regarding reproducibility, the authors report the results with a standard error over 20 runs, therefore we consider a result reproduced if our result is in the given interval. We could reproduce only 3 out of 14 results for the proposed model and 2 out of 28 results for the baselines. In the end, we could reproduce the 11.9% of the experiments. Since we could not set all the hyperparameters, using different values for them can be one possible explanation for these results. We consider the paper exercisable since we could execute more than the 75% of the experiments, but given the low number of reproduced results, we cannot consider the paper reproducible.

3.3 Reinforcement Knowledge Graph Reasoning for Explainable Recommendation

The ***Policy-Guided Path Reasoning (PGPR)*** algorithm was proposed at *SIGIR'19* by the paper *"Reinforcement Knowledge Graph Reasoning for Explainable Recommendation"* [74]. The authors propose to cast the recommendation problem as an MDP over a knowledge graph defined on users, items, and their relationships. Then, they use an RL approach where the agent starts from a given user and learns to navigate to reach the potential items of interest. The resulting path on the graph is leveraged to explain why the item is recommended to the user. Indeed, the main challenge that this paper highlights is how to assess the relevance of an item for a user. For this reason, a soft reward strategy on large and heterogeneous data is exploited.

All the four Amazon datasets used in the paper were publicly available: two were directly provided inside the GitHub repository of the project[13] and the other two were provided in a Google Drive repository by the authors (the link is provided in the artifacts). In this regard, we remark that, following the ACM Badges definitions, a personal Google Drive repository is not considered a suitable repository. We were able to use all datasets, so there is an appropriate pre-processing code for each of them. The hardware configuration used to run the experiments is not provided either in the paper or in the GitHub repository, while the software dependencies are reported in

[12] https://github.com/jiaqima/Off-Policy-2-Stage.
[13] https://github.com/orcax/PGPR.

Table 4. Number of Exercisable and Reproducible Experiments for the Paper *"Reinforcement Knowledge Graph Reasoning for Explainable Recommendation"* [74]

	Exercisable	Reproducible
Proposed Algorithm(s)	100.0% (16/16)	50.0% (8/16)
Baselines	0.0% (0/112)	0.0% (0/112)
Total	12.5% (16/128)	6.3% (8/128)

the GitHub repository of the original paper in a file named *requirements.txt*. All the values of the hyperparameters used for the experiments were reported in the paper, and there is also an easy way to set all of them.

We were able to execute all 16 experiments of the proposed algorithm but, since the code for the baselines was not shared, we could not execute and reproduce the related experiments. Therefore, as reported in Table 4, the number of the executable experiments is 16 out of 128 corresponding to the 12.5%. Since the authors did not report any interval for the results, we consider a result to be reproduced if the difference between the original and our results is less than 5.0%. Therefore, we reproduced the 50.0% (8/16) of the results, if we consider the results of the proposed algorithm, and the 6.3% (8/128) of the results, if we consider all the experiments including the baselines. In this case, the paper is exercisable if we consider only the proposed algorithm (100% of exercisable experiments), but we cannot consider it exercisable w.r.t. all the experiments (12.5% of exercisable experiments). Furthermore, given the low number of reproduced results, we could not consider the paper reproducible.

3.4 Generative Adversarial User Model for Reinforcement Learning Based Recommendation System

In *"Generative Adversarial User Model for Reinforcement Learning Based Recommendation System"* [14], published at *ICML'19*, the algorithms *GAN user model with position weight (GAN-PW)* and *GAN user model with LSTM (GAN-LSTM)* were proposed. The authors suggest to mimic the user behaviour and learning the reward function with a GAN. Both the behavioural model and the associated reward function are learned via a mini-max optimization. Leveraging the user model as a simulator, they develop a new *Cascading **Deep Q-Network** (**DQN**)* algorithm to obtain a recommendation policy that can efficiently handle a large number of candidate items. Experiments are performed on a plethora of real-world dataset to evaluate whether an RL GAN-based model leads to a more reliable user behavioural model and, hence, increase reward and click-through-rate, compared to the standard RS model.

The experiments include six different datasets: MovieLens, LastFM, Yelp, Taobao, YooChoose, and Ant Financial. However, only the Yelp, Taobao, and YooChoose datasets are provided in the GitHub repository of the original paper.[14] Furthermore, the authors stated that the Taobao dataset provided in the GitHub repository is only a part of the original dataset. Regarding the other three datasets, the Ant Financial dataset is private and cannot be disclosed, while the exact version of the MovieLens dataset is definitely lost. Finally, we asked the authors to provide the LastFM dataset, and they replied by sending a Dropbox link.[15] Again, we underline that a personal Dropbox repository is not considered as a suitable repository, following the ACM badges guidelines. The code necessary to pre-process the datasets is provided within the repository with a script file to

[14] https://github.com/xinshi-chen/GenerativeAdversarialUserModel.
[15] https://www.dropbox.com/s/a6j9clyu40ya1y6/lastfm_prepocessed_len_10.txt.zip?dl=0.

Table 5. Number of Exercisable and Reproducible Experiments for the Paper *"Generative Adversarial User Model for Reinforcement Learning Based Recommendation System"* [14]

	Exercisable	Reproducible
Proposed Algorithm(s)	33.3% (8/24)	8.3% (2/24)
Baselines	0.0% (0/84)	0.0% (0/84)
Total	7.4% (8/108)	1.9% (2/108)

pre-process all the shared datasets at once. The Hardware and Software configurations are not reported in the paper, but a setup file is provided in the GitHub repository to clone and install the artifacts and the necessary third-party software. The hyperparameters used in the experiments of the original paper were not reported. We asked the authors for them, but they replied that these were lost.

Table 5 reports the results in terms of the number of exercisable and reproducible experiments. In this case, the authors did not share the code for the baseline algorithms. Regarding the proposed algorithm, we could execute the experiments only on the Yelp and YooChoose datasets, i.e., 8 out of 24 results. We were unable to execute the experiments on the LastFM dataset due to an error in a Python package (*pickle*[16]) and we did not execute the experiments on the Taobao dataset since it is a different version of the dataset. We could only reproduce 2 out of 24 results for the proposed algorithm if we take into account the standard error reported by the authors in the original paper. We want to stress that the differences between our and the original results can be explained with a different hyperparameter set. We consider the paper neither exercisable nor reproducible since we could only execute the 7.4% and reproduce the 1.9% of the experiments.

3.5 Exact-K Recommendation Via Maximal Clique Optimization

The algorithm *Exact-k Recommendation* was proposed in the paper *"Exact-K Recommendation via Maximal Clique Optimization"* [24], published in *KDD '19*. The algorithm is a Graph Attention Networks with a Multi-head Self-attention encoder and a decoder with an attention mechanism. In order to efficiently train the model, the authors used the so-called RL from Demonstrations method, which combines the benefits of behaviour cloning and RL. The model is adopted to solve the learning-to-rank problem, and it is compared against baselines in qualitative and quantitative terms.

The experiments in the paper are performed on two datasets: Movielens 100k and Taobao. While the former was publicly available, and is included in the repository,[17] the latter was collected by the authors and is not available. The pre-processing code for the Movielens dataset is automatically executed before the algorithm's training phase. The hardware and software configurations are not shared within the paper, and also in the artifacts there is no reference to the software environment used. Since the algorithm's hyperparameters are not reported, neither in the paper nor in the repository, we used the default values reported in the artifacts.

In the original paper, the metrics are *Precision@4*, *Precision@10*, *HR@4,* and *HR@10*. However, in the artifacts, only the *Precision@4* metric is available. We could only execute 1 experiment out of 6 for the proposed algorithm and 0 out of 24 for the baselines since the authors did not share the code of the baselines. The numbers are reported in Table 6. Regarding reproducibility, the unique result we obtained has a difference w.r.t. the original one that is greater than the 5%, so we consider

[16]https://docs.python.org/3/library/pickle.html.
[17]https://github.com/pangolulu/exact-k-recommendation.

Table 6. Number of Exercisable and Reproducible Experiments for Paper *"Exact-K Recommendation via Maximal Clique Optimization"*[24]

	Exercisable	Reproducible
Proposed Algorithm(s)	16.7% (1/6)	0.0% (0/6)
Baselines	0.0% (0/24)	0.0% (0/24)
Total	3.3% (1/30)	0.0% (0/30)

Table 7. Number of Exercisable and Reproducible Experiments for the Paper *"Model-Based Reinforcement Learning with Adversarial Training for Online Recommendation"* [4]

	Exercisable	Reproducible
Proposed Algorithm(s)	100.0% (2/2)	(0/2 - 2/2)
Baselines	0.0% (0/14)	0.0% (0/14)
Total	12.5% (2/16)	0.0% (0/16) - 12.5% (2/16)

it unreproducible. We consider the paper neither exercisable nor reproducible since we could only execute the 3.3% and reproduce the 0% of the experiments.

3.6 Model-Based Reinforcement Learning with Adversarial Training for Online Recommendation

The paper *"Model-Based Reinforcement Learning with Adversarial Training for Online Recommendation"* [4], published at *NeurIPS '19*, describes the algorithm *IRecGAN*. The paper introduces a model-based RL algorithm which models the user–agent interaction for offline policy learning via a GAN. A discriminator is used in order to reduce bias in the user behavioural model and policy. Indeed, the main goal of this paper is to achieve an effective tradeoff between efficient offline adversarial learning and bias/noise reduction via user behavioural modelling. To this end, the presented model is evaluated through both theoretical analysis and empirical evaluation.

The algorithm is evaluated on the *CIKM Cup 2016* dataset that is possible to download, already pre-processed, from a link shared in the artifacts. The hardware and software configurations are not reported in the paper, but a shortlist of the main software dependencies is given in the README file inside the open-source artifacts repository.[18] In the original paper, the hyperparameters for the algorithm are reported only in the section of the experiments on the simulated data, and we assumed that these are the same used for the real-world data.

In our experiments, we ran out of memory after 77 epochs of training, so our results could not be directly compared with the original paper's results. However, we consider the 2 experiments with the proposed algorithm as executable, but the code for the baselines is not shared and therefore none of the 14 experiments with the baselines could be executed. Table 7 reports these results. In terms of reproducibility, even assuming that we could exactly reproduce the two results obtained by the proposed algorithm, the reproducibility would be equal to the 12.5%. Therefore, the paper is neither exercisable nor reproducible.

3.7 Self-Supervised Reinforcement Learning for Recommender Systems

The paper *"Self-Supervised Reinforcement Learning for Recommender Systems"* [77], published in *SIGIR '20*, introduces two methods, namely ***Self-Supervised Q-learning (SQN)*** and ***Self-Supervised***

[18]https://github.com/JianGuanTHU/IRecGAN.

Table 8. Number of Exercisable and Reproducible Experiments for the Paper *"Self-Supervised Reinforcement Learning for Recommender Systems"* [77]

	Exercisable	Reproducible
Proposed Algorithm(s)	100.0% (192/192)	0.0% (0/192)
Baselines	100.0% (96/96)	0.0% (0/96)
Total	100.0% (288/288)	0.0% (0/288)

Actor-Critic (SAC) to augment existing RSs algorithms. Specifically, this approach adds two output layers: one for self-supervised learning, to provide a strong gradient signal for parameter updates, and the other for RL, to act as a regularizer to guide the supervised layer. The authors proposed to add the SQN and the SAC methods to the following baseline algorithms: GRU, Caser, NltNet, and SASRec. For example, with GRU-SQN, we will refer to the GRU algorithm with the addition of the SQN layer. In this article, we will use the name *RL4REC* to identify this work.

The experiments in the paper are performed on two publicly available datasets: *RecSys Challenge '15 (RC15)* and *RetailRocket*, which are provided within the shared artifacts in a Google Drive repository.[19] The datasets are also already pre-processed. The hardware and software configurations used to run the experiments are not reported either in the paper or in the artifacts. However, the main hyperparameters of the algorithms are reported for all the algorithms.

Table 8 reports the results of the experiments. We could execute all the algorithms by running the provided python files, and therefore, we consider the paper as exercisable. In terms of reproducibility, it was impossible to fully reproduce any of the experiments, because none of the files execute on the test set. Instead, the provided artifacts only cyclically compute results on the validation sets after every 2000th round of training. Thus, no results on the two test sets from the employed datasets are returned. In order to be consistent with the ACM definition of reproducibility [1] and our procedure applied for all other papers, we did not extend the provided code to perform measurements on the test sets on our own.

4 DISCUSSION

In this section, we discuss the experimental results shown in the previous section. In Section 4.1, we propose an analysis of the shared artifacts based on the variables introduced in Section 2.2. Finally, in Section 4.2, we assign the badges defined by ACM to the papers with open access code.

4.1 Reproducibility Variables Analysis

In this section, we give more details about the variables, introduced in Section 2.2, used to analyse the reproducibility status of the papers with available artifact. In Table 9, we report if these papers fulfil the requirements for each variable while Figure 1 reports the related histogram. In the following list, we report the values that the variables can take and their respective semantics.

- Dataset (*Data*)
  (a) **Yes:** All the datasets for the considered paper are publicly available
  (b) **Partial:** A subset of the datasets for the considered paper are publicly available
  (c) **No:** None of the datasets for the considered paper are publicly available
- Preprocessing Code (*Pre-proc*)
  (a) **Yes:** The preprocessing code is available for all the datasets used

[19] https://drive.google.com/file/d/1nLL3_knhj_RbaP_IepBLkwaT6zNIeD5z/view.

Table 9. Summary of the Variables Defined in Section 2.2 for Every Paper with Open Access Artifacts

Paper	Data	Pre-proc	HW spec	SW dep	Algorithm	Hyperpar	Exp code
IRecGAN [4]	Yes	Yes	No	No	Yes	Yes	Yes
TPGR [9]	Yes	No	Yes	No	Yes	No	Unknown
Top-k [10]	No	No	No	Yes	Unknown	No	Unknown
GAN-LSTM [14]	Partial	Yes	No	Yes	Yes	No	Yes
Exact-k [24]	Partial	Partial	No	No	Yes	No	Partial
2-Stage [50]	Yes	Partial	No	Yes	Yes	Yes	Yes
KGPolicy [71]	Yes	Yes	No	Yes	Yes	Yes	Yes
PGPR [74]	Yes	Yes	No	Yes	Yes	Yes	Yes
RL4REC [77]	Yes	Yes	No	No	Yes	Yes	Partial
HRL [81]	Yes	No	Yes	No	Unknown	Yes	Unknown
KGQR [89]	Yes	Yes	No	No	Yes	Yes	Not Working
FMTCS [90]	Yes	Yes	Yes	No	Not Working	Partial	Unknown
Dyna-Q [91]	Yes	No	No	No	Not Working	Yes	Unknown

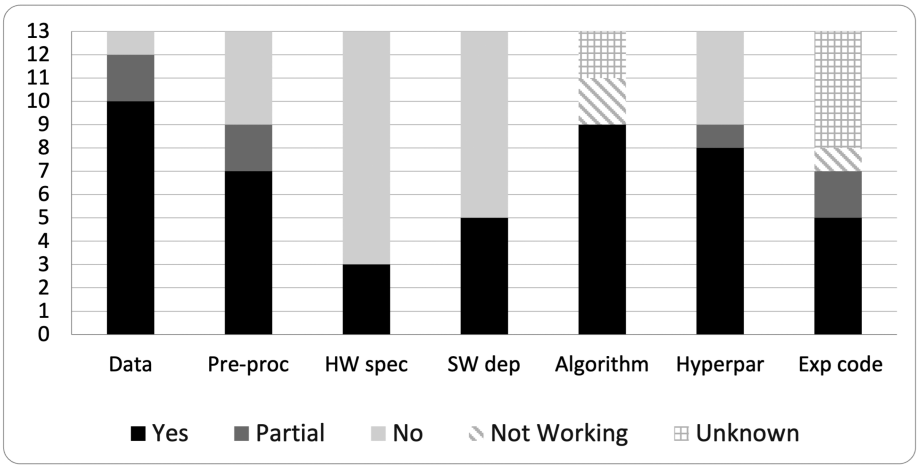

Fig. 1. Summary of the values taken by each variable.

(b) **Partial:** The preprocessing code is available only for a subset of the datasets used
(c) **No:** The preprocessing code is not available

— Hardware Specifications (*HW spec*)
(a) **Yes:** The hardware setup is reported
(b) **No:** The hardware setup is not reported

— Software Dependencies (*SW dep*)
(a) **Yes:** The software dependencies are specified
(b) **No:** The software dependencies are not specified

— Algorithm Implementation (*Algorithm*)
(a) **Yes:** The implementation of the proposed algorithm is reported
(b) **Unknown:** We are unable to test the method source code because the dataset or the preprocessing code are not included in the artifacts
(c) **Not Working:** The implementation of the proposed algorithm is reported but not working
(d) **No:** The implementation of the proposed algorithm is not reported

— Algorithm Hyperparameters (*Hyperpar*)
(a) **Yes:** All the main hyperparameters are reported

(b) **Partial:** Only a subset of the main hyperparameters are reported
(c) **No:** None of the hyperparameters are reported

— Experiment Code (*Exp code*)
(a) **Yes:** The code to reproduce the experiments is publicly available
(b) **Partial:** Only a part of the code to reproduce the experiments is publicly available
(c) **Unknown:** We are unable to test the experiment code because the dataset, pre-processing or algorithm implementation are not present
(d) **Not Working:** The code to reproduce the experiments is publicly available but not working
(e) **No:** The code to reproduce the experiments is not publicly available

Dataset. The variable *Dataset* takes value *Yes* in the 76.9% of the cases (10 out of 13 cases), value *Partial* in the 15.4% of the cases (2 out of 13), and value *No* in the 7.7% of the cases. The datasets used in the experiments are one of the most important things that should be shared to ensure reproducibility, since without the datasets the models cannot be trained and experiments cannot be reproduced. In this regard, we acknowledge that in 12 out of 13 papers, at least one experiment for each paper is performed on a publicly available dataset.

Preprocessing Code. However, sharing or providing the link to the datasets used in the experiments is not enough to ensure that other researchers can reproduce our work. It is also necessary to share the code necessary to pre-process the raw data to be used with the shared implementation of the algorithm. In this regard, the *Preprocessing Code* variable takes value *Yes* in the 53.8% of the cases, value *Partial* in the 15.4% of the cases, and in the 30.8% of the cases takes value *No*. Therefore, in the 46.2% of the cases, it is not possible to reproduce all the experiments only because the preprocessing code or the already formatted datasets are not shared.

Hardware Specifications and Software Dependencies. Regarding *Hardware Specifications* and *Software Dependencies*, these take value *Yes* only in the 23.1% and in the 38.5% of the cases, respectively. Even if the *Hardware specification* seems of limited relevance for a reproducibility experiment, this aspect can lead to huge variance in the results, as studied in [53]. For example, due to floating-point arithmetic [23], different **Graphics Processing Units (GPUs)** and different drivers. Similarly, also the *Software Dependencies* are fundamental to ensure reproducibility since experiments that once worked with specific dependency versions can behave differently or even stop working with different versions.

Algorithm Implementation. In papers that propose new algorithms, the most important part of the work is the description of the algorithm. However, the implementation of the algorithm is as critical as the methods section in the paper since the code is likely to be a better description of the work than the methods section itself. Furthermore, when data cannot be shared, the implementation of the algorithm could at least give the possibility to test the solution in different contexts and datasets. In our analysis, we observe that the *Algorithm Implementation* variable takes value *Yes* in the 69.2% of the cases. However, considering all the initial 60 papers, the algorithm code is shared and works only in 9 cases out of 60 (15.0%). This percentage is much higher than the 8% reported in [27] about general AI literature in top conferences in 2013 and 2016. Therefore, our study shows an increasing trend in releasing open-source code, at least in the RL for RSs community w.r.t. the general AI literature. Finally, we want to stress that describing the algorithm, even through a pseudo-code, is not enough to ensure reproducibility since it is unlikely that other researchers could exactly replicate the implementation from the method description, and several small details may lead to different results.

Table 10. Summary of ACM Badges Assigned to the Papers According to Our Analysis

Badge	Badge Name	Awarded Papers
	Artifacts Available	[4, 14, 24, 50, 71, 74, 90]
	Artifacts Evaluated-Functional	[50, 77]
	Artifacts Evaluated-Reusable	⊘
	Artifacts Reproduced	⊘

Algorithm Hyperparameters. Another important detail that heavily impacts the performance are the algorithm's hyperparameters. However, we note that not all the works report the complete list of the hyperparameters used in each experiment since the *Algorithm Hyperparameters* variable takes value *Yes* in the 61.5% of the cases and value *Partial* in the 7.7% of the cases. In the remaining 30.8% of the cases, we were forced to use the default values setted in the artifacts. Searching for the best combination of hyperparameter usually requires an incredible amount of time and resources, therefore it is almost infeasible for other research groups to reproduce this search for the sole purpose of reproducing experiments from other works. Therefore, the authors should report the values of the hyperparameters since, as we saw in our experiments, it is easy to obtain different results with different hyperparameters.

Experiment Code. Finally, regarding the *Experiment Code* variable, this takes value *Yes* in the 38.5% of the cases, and in the 15.3% of the cases it takes value *Partial*. In the 7.7% of the cases, this variable takes value *Not Working*. In the 38.5% of the cases, we were unable to test this part of the provided artifact, because the algorithm code or the preprocessing code did not work. Referring to the whole set of papers considered, the *Experiment Code* is available and works for the 8.3% (5/60) of the papers, which is slightly higher than the 6% reported in [27]. This aspect is not fundamental to ensure reproducibility but could help other researchers to avoid methodological errors during the process that could lead to disputes over results reported in papers.

4.2 ACM Artifact Review and Badging

In this section, we classify the selected papers with open-access artifacts using the ACM Badges. However, while the variables defined above are a quantitative measure and less prone to subjective evaluation, this badge assignment is a qualitative measure, and therefore is more prone to different evaluations by different viewers. Table 10 shows the papers and the assigned badges. The reproducibility variables defined in Section 2.2 allow a fine-grained analysis, while the ACM Badges evaluate the paper in a more global way since to achieve a badge, a paper must satisfy different requirements. However, our variables take into account some factors that the ACM badges do not consider and that, in our opinion, heavily contribute to the exact reproducibility of the paper, such as hardware specifications, software dependencies, and algorithm hyperparameters.

The definition of the *Artifacts Available* badge given by ACM is "Author-created artifacts relevant to this paper have been placed on a publicly accessible archival repository [...]. In all cases,

repositories used to archive data should have a declared plan to enable permanent accessibility. Personal web pages are not acceptable for this purpose". However, since GitHub is a standard used in many research communities, we relaxed this definition to include it as a suitable repository. For the assignment of this badge, we consider the artifacts to be sufficient if they share (i) at least one of the datasets used in the experiments, (ii) the relative pre-processing code, (iii) the algorithm implementation, and (iv) the code to perform the experiments. These requirements correspond to four of the reproducibility variables described above. The papers that received this badge are reported in the corresponding row of Table 10. We include [90] even if the algorithm implementation did not work because, according to the ACM definition, "Artifacts do not need to have been formally evaluated in order for an article to receive this badge". Instead, we do not include [77] and [89] since they make available the artifacts using Google Drive and Dropbox personal repositories. Furthermore, we do not award [9, 81, 91] since they do not share the pre-processing code and [10] since they performed online experiments (hence there are no shared datasets and pre-processing code, and it is impossible to reproduce the experiments).

Regarding the *Artifacts Evaluated - Functional* badge, we award the papers in the corresponding row of Table 10. According to the ACM definitions, the papers must be *consistent* (the artifacts are relevant to the paper), *complete* (all components relevant to the paper are included), and *exercisable* (the code can be successfully executed). In other terms, the artifacts satisfy the same requirements as the *Artifacts Available* badge, but with the addition that the pre-processing, the algorithm, and the experiment code are executable. Given the previous analysis, we consider only two papers among the seven that receive the *Artifacts Available* badge as functional because the others enable to execute less than the 75% of the experiments. The main problem is that only a few papers provide the code to execute the experiments of the baselines, hence it is impossible to execute several experiments.

The *Artifacts Evaluated - Reusable* badge is assigned to papers that share artifacts that "have all the qualities of the *Artifacts Evaluated - Functional* level, but, in addition, they are very carefully documented and well-structured to the extent that reuse and repurposing is facilitated". In none of the papers that have passed the *Artifacts Evaluated - Functional,* we can find a very careful documentation designed to facilitate reuse and repurposing. To achieve this badge, the code must be written taking into account a general input, instead of designing the algorithm for a specific dataset, and include well-written documentation that makes it possible to write extensions to the code.

Finally, the *Results Reproduced* badge is assigned to papers for which "The main results of the paper have been obtained in a subsequent study by a person or team other than the authors, using, in part, artifacts provided by the author". We relax the constraints to obtain this badge considering if, at least for part of the dataset used, we were able to reproduce the results. Furthermore, ACM specifies that "In each case, exact replication or reproduction of results is not required, or even expected. Instead, the results must be in agreement with a tolerance deemed acceptable for experiments of the given type. In particular, differences in the results should not change the main claims made in the paper". According to this definition and our previous analysis, we conclude that none of the collected papers can be considered reproducible since none of these achieve more than the 12.5% of reproduced results.

4.3 How to Enhance Reproducibility

In this article, according to the ACM definitions, the artifacts comprise several components. However, not all these components have the same importance to enable reproducibility, although each one is important to ensure the exact reproducibility of the results. The analysis proposed in the above sections clearly highlights that, even for the few papers with open-access artifacts, there are

some factors that limit the reproducibility of the works. For example, although the vast majority of the dataset used is freely accessible, it is not uncommon to have no access to the preprocessing code needed to reproduce the experiments. Similarly, in some cases, also the implementation of the proposed algorithm(s) presents some problems that make it impossible to run the experiments. In this section, we want to present some good practices to improve reproducibility, following the ideas proposed in [31] and [54].

Fundamentals. The fundamental artifacts that must be shared to enable the reproducibility of the experiments are (i) the raw form of the datasets, (ii) the pre-processing code, (iii) the algorithm implementation, and (iv) the code to perform the experiments. Furthermore, it is necessary to provide all these artifacts in a "publicly accessible archival repository" such as *Figshare* (https://figshare.com/), *Dryad* (https://datadryad.org/), or *Zenodo* (https://zenodo.org). In this article, we consider valid also GitHub personal repositories, but these are not recognized by ACM since "personal web pages are not acceptable for this purpose". In case the dataset is already publicly available, the authors should indicate the exact source from which they downloaded the version of the dataset used for the experiment, while in the case of a private dataset, it should be made publicly available in raw form. However, we recognize that not all datasets can be made public for reasons of privacy or data use agreements. The solution to this problem may be different for each specific case, but authors should try to find the best tradeoff between privacy and reproducibility to improve confidence in their results.

Important. While the recommendations in the previous paragraph are fundamental to enable reproducibility, they are not enough to ensure exact reproducibility. Several analysed papers fulfilled the above recommendations, but we still obtained different results on the same experiments, performed with the same code on the same datasets. These differences can result from many factors, e.g., different versions of third-party software and hardware characteristics. To this end, the use of dependency management tools, such as *Packrat* (https://rstudio.github.io/packrat/) and *Conda* (https://conda.io/), or containerization, using *Docker* (https://www.docker.com/), can help to resolve this problem. Regarding the hardware, it is good practice to report the main characteristics of the hardware used to perform the experiments, so that a third-party analyst can take any differences into account. Among the main information there are the operating system version, the model and number of **Central Processing Units (CPUs)** and GPUs, and the amount of CPU RAM and GPU RAM required. Furthermore, the authors should also share the hyperparameters of the proposed algorithm and the baselines, directly in the paper or at least within the artifacts, since these heavily influence the performance of the algorithms. Finally, randomization is another key aspect that determines the results of the experiments in machine learning analyses. Indeed, it is good practice to set and share the seed of the pseudorandom number generators to ensure reproducible results.

Additionals. In the previous paragraphs, the authors should provide all the steps the authors took to use the data, from the raw data to the input given to the algorithm, and the necessary documentation to use them. However, to facilitate other researchers to reproduce the experiments, the entire analysis (including downloading data, pre-processing data, training models, producing output, and generating the results shown in the paper) should be reproducible with a single command. A simple shell script can accomplish this goal, but workflow management software, like Snakemake (https://snakemake.readthedocs.io/en/stable/) or Nextflow (https://www.nextflow.io/) provides several advantages without extra work from the authors. Finally, authors should also store and publish intermediate outputs to allow reproducibility even in case the algorithm requires huge computational resources and time.

5 CONCLUSIONS

Science requires trust. Since it is impossible for a scientist to reproduce the results of every paper, we need a way to ensure that machine learning analyses can be trusted. To this end, machine learning scientists must prioritize computational reproducibility to allow third-party researchers to examine in depth the works and become worthy of trust. In this article, we investigated the state of reproducibility in the RL & RS literature to assess whether it is possible to reproduce the results of papers that proposed a new RL algorithm to solve the RS problem. to the first page footnote.

We collected 60 papers that proposed a new RL algorithm in RS, and the 21.7% (13 out of 60) of them shared some artifact related to the paper. Then, using the set of variables defined through this paper and the definition of ACM badges, we deepened these 13 papers to investigate if these could effectively be reproduced. Looking at the ACM badges assigned in Section 4.2, we assessed that only the 11.7% (7/60) of the papers could receive the *Artifact Available* badge only because we considered GitHub as a suitable repository against the ACM definition. Furthermore, considering a paper functional only if we could execute at least the 75% of the experiments and it satisfied the other requirements, only the 3.3% (2/60) of the papers achieved the *Artifacts Evaluated-Functional* badge. Regarding the *Artifacts Evaluated-Reusable* and the *Artifacts Reproduced* badges, none of the analysed papers could receive them. None of the papers were intended to allow the reuse and repurposing of the code, or allowed reproducing more than the 75% of the results. However, to obtain one of the ACM badges, a paper must fulfil many requirements. Therefore, the lack of a single requirement, even if we recognize that all requirements are important to achieve reproducibility, results in total failure. Therefore, we also analysed the papers by defining a set of variables that aim to inspect the single aspects involved in reproducibility. We found that the main missingness in the studied papers concerned the sharing of hardware configuration, software dependencies, and hyperparameters values.

Our study clearly depicts that future works in the RL & RS field must pay more attention to the reproducibility side in order to enhance the scientific progress properly. To this end, we proposed a list of recommendations to improve reproducibility, providing suggestions to fulfil all aspects that guarantee it.

REFERENCES

[1] Xiangyu Zhao, Liang Zhang, Long Xia, Zhuoye Ding, Dawei Yin, and Jiliang Tang. 2019. Deep Reinforcement Learning for List-wise Recommendations. arXiv:1801.00209 [cs.LG].

[2] M. Mehdi Afsar, Trafford Crump, and Behrouz Far. 2021. Reinforcement learning based recommender systems: A survey. *arXiv preprint arXiv:2101.06286* 1 (2021).

[3] M. Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *ACM Computing Surveys* 55, 7, Article 145 (Dec. 2022), 38 pages.

[4] Xueying Bai, Jian Guan, and Hongning Wang. 2019. A model-based reinforcement learning with adversarial training for online recommendation. In *Advances in Neural Information Processing Systems*. H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32, Curran Associates, Inc., Vancouver.

[5] Richard Bellman. 1957. A Markovian decision process. *Journal of Mathematics and Mechanics* 6, 5 (1957), 679–684.

[6] Robin Burke. 1999. Integrating knowledge-based and collaborative-filtering recommender systems. In *Proceedings of the Workshop on AI and Electronic Commerce*. AAAI, 69–72.

[7] Walter Carrer-Neto, María Luisa Hernández-Alcaraz, Rafael Valencia-García, and Francisco García-Sánchez. 2012. Social knowledge-based recommender system. Application to the movies domain. *Expert Systems with Applications* 39, 12 (2012), 10990–11000.

[8] Jia-Wei Chang, Ching-Yi Chiou, Jia-Yi Liao, Ying-Kai Hung, Chien-Che Huang, Kuan-Cheng Lin, and Ying-Hung Pu. 2019. Music recommender using deep embedding-based features and behavior-based reinforcement learning. *Multimedia Tools and Applications* 80, 26 (2019), 1–28.

[9] Haokun Chen, Xinyi Dai, Han Cai, Weinan Zhang, Xuejian Wang, Ruiming Tang, Yuzhou Zhang, and Yong Yu. 2019. Large-scale interactive recommendation with tree-structured policy gradient. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 01 (2019), 3312–3320.

[10] Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, and Ed H. Chi. 2019. Top-k off-policy correction for a REINFORCE recommender system. In *Proceedings of the 12th ACM International Conference on Web Search and Data Mining*. ACM, New York, NY, 456–464.

[11] Minmin Chen, Bo Chang, Can Xu, and Ed H. Chi. 2021. User response models to improve a REINFORCE recommender system. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. ACM, New York, NY, 121–129.

[12] Shi-Yong Chen, Yang Yu, Qing Da, Jun Tan, Hai-Kuan Huang, and Hai-Hong Tang. 2018. Stabilizing reinforcement learning in dynamic environment with application to online recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, New York, NY, 1187–1196.

[13] Xiaocong Chen, Chaoran Huang, Lina Yao, Xianzhi Wang, Wei Liu, and Wenjie Zhang. 2020. Knowledge-guided deep reinforcement learning for interactive recommendation. In *Proceedings of the 2020 International Joint Conference on Neural Networks*. Vol. 1, IEEE, 1–8.

[14] Xinshi Chen, Shuang Li, Hui Li, Shaohua Jiang, Yuan Qi, and Le Song. 2019. Generative adversarial user model for reinforcement learning based recommendation system. In *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97, PMLR, 1052–1061.

[15] Xiaocong Chen, Lina Yao, Julian McAuley, Guanglin Zhou, and Xianzhi Wang. 2021. A survey of deep reinforcement learning in recommender systems: A systematic review and future directions. *arXiv preprint arXiv:2109.03540* 1 (2021).

[16] Xiaocong Chen, Lina Yao, Julian McAuley, Guanglin Zhou, and Xianzhi Wang. 2023. Deep reinforcement learning in recommender systems: A survey and new perspectives. *Knowledge-Based Systems* 264, 1 (2023), 110335.

[17] Xiaocong Chen, Lina Yao, Aixin Sun, Xianzhi Wang, Xiwei Xu, and Liming Zhu. 2021. Generative inverse deep reinforcement learning for online recommendation. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*. ACM, New York, NY, 201–210.

[18] Sungwoon Choi, Heonseok Ha, Uiwon Hwang, Chanju Kim, Jung-Woo Ha, and Sungroh Yoon. 2018. Reinforcement learning based recommender system using biclustering technique. (2018). arXiv:1801.05532 [cs.IR].

[19] Gabriel Dulac-Arnold, Richard Evans, Hado van Hasselt, Peter Sunehag, Timothy Lillicrap, Jonathan Hunt, Timothy Mann, Theophane Weber, Thomas Degris, and Ben Coppin. 2016. Deep reinforcement learning in large discrete action spaces. (2016). arXiv:1512.07679 [cs.AI].

[20] Jun Feng, Heng Li, Minlie Huang, Shichen Liu, Wenwu Ou, Zhirong Wang, and Xiaoyan Zhu. 2018. Learning to collaborate: Multi-scenario ranking via multi-agent reinforcement learning. In *Proceedings of the 2018 World Wide Web Conference*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 1939–1948.

[21] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys'19 Copenhagen, Denmark)*, Association for Computing Machinery, New York, NY, 101–109.

[22] Rong Gao, Haifeng Xia, Jing Li, Donghua Liu, Shuai Chen, and Gang Chun. 2019. DRCGR: Deep reinforcement learning framework incorporating CNN and GAN-based for interactive recommendation. In *Proceedings of the 2019 IEEE International Conference on Data Mining (ICDM'19)*, Vol. 1. IEEE, 1048–1053.

[23] David Goldberg. 1991. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys* 23, 1 (1991), 5–48.

[24] Yu Gong, Yu Zhu, Lu Duan, Qingwen Liu, Ziyu Guan, Fei Sun, Wenwu Ou, and Kenny Q. Zhu. 2019. Exact-k recommendation via maximal clique optimization. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'19 Anchorage, AK)*, ACM, New York, NY, 617–626.

[25] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.

[26] Claudio Greco, Alessandro Suglia, Pierpaolo Basile, and Giovanni Semeraro. 2017. Converse-et-impera: Exploiting deep learning and hierarchical reinforcement learning for conversational recommender systems. In *AI*IA 2017 Advances in Artificial Intelligence*. Springer International Publishing, Cham, 372–386.

[27] Odd Erik Gundersen and Sigbjørn Kjensmo. 2018. State of the art: Reproducibility in artificial intelligence. *Proceedings of the AAAI Conference on Artificial Intelligence* 32, 1 (2018), 1644–1651.

[28] Jianhua Han, Yong Yu, Feng Liu, Ruiming Tang, and Yuzhou Zhang. 2019. Optimizing ranking algorithm in recommender system via deep reinforcement learning. In *Proceedings of the 2019 International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM'19)*, Vol. 1, IEEE, 22–26.

[29] Naieme Hazrati and Francesco Ricci. 2022. Recommender systems effect on the evolution of users' choices distribution. *Information Processing & Management* 59, 1 (2022), 102766.

[30] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW'17 Perth, Australia)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 173–182.

[31] Benjamin J. Heil, Michael M. Hoffman, Florian Markowetz, Su-In Lee, Casey S. Greene, and Stephanie C. Hicks. 2021. Reproducibility standards for machine learning in the life sciences. *Nature Methods* 18, 10 (2021), 1132–1135.

[32] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based recommendations with recurrent neural networks. (2016). arXiv:1511.06939 [cs.LG].

[33] Daocheng Hong, Yang Li, and Qiwen Dong. 2020. Nonintrusive-sensing and reinforcement-learning based adaptive personalized music recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'20 Virtual Event, China)*. ACM, New York, NY, 1721–1724.

[34] Binbin Hu, Chuan Shi, and Jian Liu. 2017. Playlist recommendation based on reinforcement learning. In *Intelligence Science I*. Springer International Publishing, Cham, 172–182.

[35] Matthew Hutsonm. 2018. Artificial intelligence faces reproducibility crisis. *Science* 359, 6377 (2018), 725–726.

[36] Eugene Ie, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Morgane Lustman, Vince Gatto, Paul Covington, Jim McFadden, Tushar Chandra, and Craig Boutilier. 2019. Reinforcement learning for slate-based recommender systems: A tractable decomposition and practical methodology. (2019). arXiv:1905.12767 [cs.LG].

[37] Wacharawan Intayoad, Chayapol Kamyod, and Punnarumol Temdee. 2018. Reinforcement learning for online learning recommendation system. In *Proceedings of the 2018 Global Wireless Summit (GWS)*, Vol. 1, IEEE, 167–170.

[38] Sarika Jain, Anjali Grover, Praveen Singh Thakur, and Sourabh Kumar Choudhary. 2015. Trends, problems and solutions of recommender system. In *Proceedings of the International Conference on Computing, Communication & Automation*, Vol. 1, IEEE, 955–958.

[39] Dietmar Jannach, Markus Zanker, Alexander Felfernig, and Gerhard Friedrich. 2010. *Recommender Systems: An Introduction*. Cambridge University Press.

[40] Yu Lei and Wenjie Li. 2019. Interactive recommendation with user-specific deep reinforcement learning. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 13, 6 (2019), 1–15.

[41] Yu Lei, Hongbin Pei, Hanqi Yan, and Wenjie Li. 2020. Reinforcement learning based recommendation with graph convolutional q-network. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'20 Virtual Event, China)*. ACM, New York, NY, 1757–1760.

[42] Huizhi Liang. 2020. DRprofiling: Deep reinforcement user profiling for recommendations in heterogenous information networks. *IEEE Transactions on Knowledge and Data Engineering* 34, 4 (2020), 1723–1734.

[43] Elad Liebman, Maytal Saar-Tsechansky, and Peter Stone. 2015. DJ-MC: A reinforcement-learning agent for music playlist recommendation. In *Proceedings of the 2015 International Conference on Autonomous Agents and Multiagent Systems (AAMAS'15 Istanbul, Turkey)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 591–599.

[44] Feng Liu, Huifeng Guo, Xutao Li, Ruiming Tang, Yunming Ye, and Xiuqiang He. 2020. End-to-end deep reinforcement learning based recommendation with supervised embedding. In *Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM'20 Houston, TX)*. ACM, New York, NY, 384–392.

[45] Feng Liu, Ruiming Tang, Huifeng Guo, Xutao Li, Yunming Ye, and Xiuqiang He. 2020. Top-aware reinforcement learning based recommendation. *Neurocomputing* 417, 1 (2020), 255–269.

[46] Feng Liu, Ruiming Tang, Xutao Li, Weinan Zhang, Yunming Ye, Haokun Chen, Huifeng Guo, and Yuzhou Zhang. 2018. Deep reinforcement learning based recommendation with explicit user-item interactions modeling. *arXiv preprint arXiv:1810.12027* (2018).

[47] Feng Liu, Ruiming Tang, Xutao Li, Weinan Zhang, Yunming Ye, Haokun Chen, Huifeng Guo, Yuzhou Zhang, and Xiuqiang He. 2020. State representation modeling for deep reinforcement learning based recommendation. *Knowledge-Based Systems* 205, 1 (2020), 106170.

[48] Su Liu, Ye Chen, Hui Huang, Liang Xiao, and Xiaojun Hei. 2018. Towards smart educational recommendations with reinforcement learning in classroom. In *Proceedings of the 2018 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE)*, Vol. 1, IEEE, 1079–1084.

[49] Zhongqi Lu and Qiang Yang. 2016. Partially observable Markov decision process for recommender systems. *arXiv preprint arXiv:1608.07793* (2016).

[50] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Ji Yang, Minmin Chen, Jiaxi Tang, Lichan Hong, and Ed H. Chi. 2020. Off-policy learning in two-stage recommender systems. In *Proceedings of the Web Conference 2020 (WWW'20 Taipei, Taiwan)*. ACM, New York, NY, 463–473.

[51] Isshu Munemasa, Yuta Tomomatsu, Kunioki Hayashi, and Tomohiro Takagi. 2018. Deep reinforcement learning for recommender systems. In *Proceedings of the 2018 International Conference on Information and Communications Technology (ICOIACT)*, Vol. 1, IEEE, 226–233.

[52] Michael J. Pazzani and Daniel Billsus. 2007. Content-based recommendation systems. In *The Adaptive Web*. Springer, 325–341.

[53] Hung Viet Pham, Shangshu Qian, Jiannan Wang, Thibaud Lutellier, Jonathan Rosenthal, Lin Tan, Yaoliang Yu, and Nachiappan Nagappan. 2021. Problems and opportunities in training deep learning software systems: An analysis of variance. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE'20 Virtual Event, Australia)*. ACM, New York, NY, 771–783.

[54] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. 2021. Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research* 22, 1 (2021), 7459–7478.

[55] Faxin Qi, Xiangrong Tong, Lei Yu, and Yingjie Wang. 2019. Personalized project recommendations: Using reinforcement learning. *EURASIP Journal on Wireless Communications and Networking* 2019, 1 (2019), 1–17.

[56] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. 2020. Neural collaborative filtering vs. matrix factorization revisited. In *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys'20 Virtual Event, Brazil)*. ACM, New York, NY, 240–248.

[57] Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. 1994. GroupLens: An open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work (CSCW'94 Chapel Hill, North Carolina)*. ACM, New York, NY, 175–186.

[58] Francesco Ricci, Lior Rokach, and Bracha Shapira. 2011. Introduction to recommender systems handbook. In *Recommender Systems Handbook*. Springer, 1–35.

[59] Herbert Robbins. 1952. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society* 58, 5 (1952), 527–535.

[60] Aleksandrs Slivkins. 2019. Introduction to multi-armed bandits. *Foundations and Trends® in Machine Learning* 12, 1–2 (2019), 1–286.

[61] Dusan Stamenkovic, Alexandros Karatzoglou, Ioannis Arapakis, Xin Xin, and Kleomenis Katevas. 2022. Choosing the best of both worlds: Diverse and novel recommendations through multi-objective reinforcement learning. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM'22 Virtual Event, AZ)*. ACM, New York, NY, 957–965.

[62] Yueming Sun and Yi Zhang. 2018. Conversational recommender system. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'18 Ann Arbor, MI, USA)*. ACM, New York, NY, 235–244.

[63] Peter Sunehag, Richard Evans, Gabriel Dulac-Arnold, Yori Zwols, Daniel Visentin, and Ben Coppin. 2015. Deep reinforcement learning with attention for slate Markov decision processes with high-dimensional states and actions. *arXiv preprint arXiv:1512.01124* (2015).

[64] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.

[65] Georgios Theocharous, Philip S. Thomas, and Mohammad Ghavamzadeh. 2015. Personalized ad recommendation systems for life-time value optimization with guarantees. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence*. AAAI, Buenos Aires, Argentina, 20.

[66] Emmanouil Vozalis and Konstantinos G. Margaritis. 2003. Analysis of recommender systems algorithms. In *Proceedings of the 6th Hellenic European Conference on Computer Mathematics & Its Applications*. 732–745.

[67] Kai Wang, Zhene Zou, Qilin Deng, Jianrong Tao, Runze Wu, Changjie Fan, Liang Chen, and Peng Cui. 2021. Reinforcement learning with a disentangled universal value function for item recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 5, 4427–4435.

[68] Lu Wang, Wei Zhang, Xiaofeng He, and Hongyuan Zha. 2018. Supervised reinforcement learning with recurrent neural network for dynamic treatment recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18 London, United Kingdom)*. ACM, New York, NY, 2447–2456.

[69] Pengfei Wang, Yu Fan, Long Xia, Wayne Xin Zhao, Shaozhang Niu, and Jimmy Huang. 2020. KERL: A knowledge-guided reinforcement learning model for sequential recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'20 Virtual Event, China)*. ACM, New York, NY, 209–218.

[70] Xiting Wang, Yiru Chen, Jie Yang, Le Wu, Zhengtao Wu, and Xing Xie. 2018. A reinforcement learning framework for explainable recommendation. In *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)*, Vol. 1, IEEE, 587–596.

[71] Xiang Wang, Yaokun Xu, Xiangnan He, Yixin Cao, Meng Wang, and Tat-Seng Chua. 2020. Reinforced negative sampling over knowledge graph for recommendation. In *Proceedings of the Web Conference 2020 (WWW'20 Taipei, Taiwan)*. ACM, New York, NY, 99–109.

[72] Yu Wang. 2020. A hybrid recommendation for music based on reinforcement learning. *Advances in Knowledge Discovery and Data Mining* 12084, 1 (2020), 91–103.

[73] Peter Wei, Stephen Xia, Runfeng Chen, Jingyi Qian, Chong Li, and Xiaofan Jiang. 2020. A deep-reinforcement-learning-based recommender system for occupant-driven energy optimization in commercial buildings. *IEEE Internet of Things Journal* 7, 7 (2020), 6402–6413.

[74] Yikun Xian, Zuohui Fu, S. Muthukrishnan, Gerard de Melo, and Yongfeng Zhang. 2019. Reinforcement knowledge graph reasoning for explainable recommendation. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'19 Paris, France)*. ACM, New York, NY, 285–294.

[75] Yilin Xiao, Liang Xiao, Xiaozhen Lu, Hailu Zhang, Shui Yu, and H. Vincent Poor. 2020. Deep-reinforcement-learning-based user profile perturbation for privacy-aware recommendation. *IEEE Internet of Things Journal* 8, 6 (2020), 4560–4568.

[76] Ruobing Xie, Shaoliang Zhang, Rui Wang, Feng Xia, and Leyu Lin. 2021. Hierarchical reinforcement learning for integrated recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 5 (2021), 4521–4528. Retrieved from https://ojs.aaai.org/index.php/AAAI/article/view/16580.

[77] Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M. Jose. 2020. Self-supervised reinforcement learning for recommender systems. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'20 Virtual Event, China)*. ACM, New York, NY, 931–940.

[78] Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M. Jose. 2022. Supervised advantage actor-critic for recommender systems. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM'22 Virtual Event, AZ, USA)*. ACM, New York, NY, 1186–1196.

[79] Tong Yu, Yilin Shen, Ruiyi Zhang, Xiangyu Zeng, and Hongxia Jin. 2019. Vision-language recommendation via attribute augmented multimodal reinforcement learning. In *Proceedings of the 27th ACM International Conference on Multimedia (MM'19 Nice, France)*. ACM, New York, NY, 39–47.

[80] Zhang Yuyan, Su Xiayao, and Liu Yong. 2019. A novel movie recommendation system based on deep reinforcement learning with prioritized experience replay. In *Proceedings of the 2019 IEEE 19th International Conference on Communication Technology*. Vol. 1, IEEE, 1496–1500.

[81] Jing Zhang, Bowen Hao, Bo Chen, Cuiping Li, Hong Chen, and Jimeng Sun. 2019. Hierarchical reinforcement learning for course recommendation in MOOCs. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 01 (2019), 435–442.

[82] Chenfei Zhao and Lan Hu. 2019. CapDRL: A deep capsule reinforcement learning for movie recommendation. In *Proceedings of the Pacific Rim International Conference on Artificial Intelligence*. Springer, Springer International Publishing, Cham, 734–739.

[83] Dongyang Zhao, Liang Zhang, Bo Zhang, Lizhou Zheng, Yongjun Bao, and Weipeng Yan. 2020. MaHRL: Multi-goals abstraction based deep hierarchical reinforcement learning for recommendations. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'20 Virtual Event, China)*. ACM, New York, NY, 871–880.

[84] Xiangyu Zhao, Long Xia, Jiliang Tang, and Dawei Yin. 2019. Deep reinforcement learning for search, recommendation, and online advertising: A survey. *ACM SIGWEB Newsletter* 2019, Spring (2019), 1–15.

[85] Xiangyu Zhao, Long Xia, Liang Zhang, Zhuoye Ding, Dawei Yin, and Jiliang Tang. 2018. Deep reinforcement learning for page-wise recommendations. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys'18 Vancouver, British Columbia, Canada)*. ACM, New York, NY, 95–103.

[86] Xiangyu Zhao, Long Xia, Lixin Zou, Hui Liu, Dawei Yin, and Jiliang Tang. 2020. Whole-chain recommendations. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM'20 Virtual Event, Ireland)*. ACM, New York, NY, 1883–1891.

[87] Xiangyu Zhao, Liang Zhang, Zhuoye Ding, Long Xia, Jiliang Tang, and Dawei Yin. 2018. Recommendations with negative feedback via pairwise deep reinforcement learning. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18 London, United Kingdom)*. ACM, New York, NY, 1040–1048.

[88] Guanjie Zheng, Fuzheng Zhang, Zihan Zheng, Yang Xiang, Nicholas Jing Yuan, Xing Xie, and Zhenhui Li. 2018. DRN: A deep reinforcement learning framework for news recommendation. In *Proceedings of the 2018 World Wide Web Conference (WWW'18 Lyon, France)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 167–176.

[89] Sijin Zhou, Xinyi Dai, Haokun Chen, Weinan Zhang, Kan Ren, Ruiming Tang, Xiuqiang He, and Yong Yu. 2020. Interactive recommender system via knowledge graph-enhanced reinforcement learning. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'20 Virtual Event, China)*. ACM, New York, NY, 179–188.

[90] Lixin Zou, Long Xia, Zhuoye Ding, Dawei Yin, Jiaxing Song, and Weidong Liu. 2019. Reinforcement learning to diversify top-n recommendation. In *Proceedings of the International Conference on Database Systems for Advanced Applications*. Springer International Publishing, Cham, 104–120.

[91] Lixin Zou, Long Xia, Pan Du, Zhuo Zhang, Ting Bai, Weidong Liu, Jian-Yun Nie, and Dawei Yin. 2020. Pseudo Dyna-Q: A reinforcement learning framework for interactive recommendation. In *Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM'20 Houston, TX, USA)*. ACM, New York, NY, 816–824.

Received 18 May 2022; revised 28 March 2023; accepted 5 April 2023